



Simulation based approach for inventory management of packaged milk at an unorganized Indian retailer: A case study

Nishit Satish Mehta¹, Anand Abraham^{1*}, and K. Nageswara Reddy²

¹ Department of Industrial and Systems Engineering, Indian Institute of Technology Kharagpur, India

² Operations Research Scientist, Bangalore, India

*anandabraham@iem.iitkgp.ac.in

Abstract. Managing inventory for perishable goods such as milk is a complex task, particularly for small unorganized retailers who handle multiple products and lack the resources to closely monitor each item or employ costly optimization tools. Such retailers typically rely on simple thumb rules or fixed-order policies to manage stock. This study develops a Discrete-Event Simulation (DES) model to replicate the daily operations of such a retailer managing multiple milk SKUs under demand uncertainty and product perishability. A Genetic Algorithm (GA) is then applied to optimize the parameters of a base-stock policy, aiming to improve ordering decisions. Results indicate that the GA-optimized policy yields marginally higher profitability and reduced lost sales compared to the traditional fixed-order approach. While the improvement is statistically significant, its magnitude remains modest, suggesting that the simple heuristics used by small retailers are relatively effective and continue to be practical for real-world supply chain operations.

Keywords: Inventory management; Genetic Algorithm (GA); fixed-order policy; based-stock policy; simulation-based optimization; perishable goods; supply chain

1 Introduction

In India, the distribution and sale of perishable goods such as milk are predominantly managed by small, unorganized retailers. These retailers typically lack access to advanced technological tools and data-driven decision-making systems, resulting in the absence of formalized inventory control or optimal ordering policies. Instead, they depend on heuristic approaches and personal intuition to manage their stock levels. The management of products with limited shelf life and constrained refrigeration capacity presents significant operational challenges, often leading to frequent instances of stock-outs and spoilage. Therefore, there exists considerable potential for research and development in enhancing inventory management practices for perishable commodities like milk within the unorganized retail sector.

In inventory management, the mathematical study of inventory policies was pioneered with the EOQ model [4]. The model had assumptions, including the absence of stockouts and the imperishability of items. To address the limitations of these models, a model that considers shelf life, such model was one of the models that opened the gates for research in this area [8]. As computational technology has advanced, simulation models have become a popular way to study and optimize complex inventory systems. Discrete-event simulation to analyze transportation and demand management in service supply chains, showing its value in handling real-world uncertainties [3]. Similar approaches are discussed in various books which outlined the foundations of simulation-based analysis [2], [6] and [1]. Classic inventory texts also highlight how such methods complement traditional inventory planning and control [9]. Further methods like Genetic Algorithm (GA) for perishable distribution network, which gave more optimized results [5]. Later methods like Discrete Event Simulation (DES) for highly perishable goods for restaurants which gave optimized results resulting a very low stockout and food wastage [10]. These studies state to us that using modern methods rather than a traditional approach for perishable goods inventory management can optimize the results.

Building on insights from these studies, this case study examines the inventory optimization of milk, a perishable product, for a small local unorganized retailer managing multiple SKUs. A discrete event simulation model is used to replicate the retailer's daily operations, considering uncertain demand, limited supply flexibility, and FIFO-based product movement, also which follows traditional fixed order policy. It is compared with an optimized base-stock policy, which uses modern methods like Genetic Algorithm for better results. This is used to understand how simple thumb rules and policies used by small retailers can be enhanced using modern optimization methods.

2 System Description

A small-scale retailer selling multiple varieties of milk is considered. Each product faces a daily demand variability and has a limited shelf life. Unsold products get accumulated in inventory and gets discarded after its expiry date. The inventory capacity is finite and shared by all milk products. To replenish inventory, product-specific orders are placed periodically (daily) with a lead time of 1 day. The relevant performance measure in this is:

1. Average profitability for each SKU
2. mean total profit per day
3. Wastage
4. Lost sales.

Simulation of retail sales is carried out for 120 days to assess system performance. Demand consumption takes place of a First-In-First-Out (FIFO) basis, ensuring older products get sold first. This is a practical step taken by retailer to prevent wastage. If the available stock is not enough for the day's demand, the remaining part is counted as lost sales, meaning it is not delivered later. Orders are placed at the end of each day and come the next morning, keeping a simple daily routine of checking, ordering, and receiving goods. The discrete-event simulation model developed captures the daily consumption, replenishment, and loss of milk products. The modeling is based on real-life observation and personal interaction with the retailer. The model takes one type of perishable product listed under the SKU list (for example, "Milk A"), each having a fixed shelf life (s_i) after which the product expires and adds to wastage cost.

The initial ordering policy used by retailers follows a fixed order policy. The proposed ordering process follows a base-stock policy that keeps a target inventory level S_i for all i SKUs after every day, the system places an order automatically to bring the stock back to this level. In the fixed-order policy used by the retailer presently, the order size stays equal to the average daily demand. The lead time (L_i) is fixed for all i SKUs, so orders made at the end of day t arrive at the start of day $t + L_i$, and there are no late or half deliveries.

Each SKU operates independently, and the demand for one product cannot be substituted or fulfilled by another. To ensure a fair comparison, the simulation begins with identical initial inventory levels for both the Genetic Algorithm (GA)-optimized policy and the fixed-order policy. The inventory holding cost is considered as a fixed value and is not calculated separately for each milk type, as multiple products are involved and isolating the exact cost attributable solely to milk is not feasible.

3 Methodology

A Discrete Event Simulation (DES) model is made to mirror the inventory cycle of the small retailer of perishable product like milk in this study. The model uses GA for optimization of S_i for each milk i along with stochastic simulation model. The simulation model and its implementation is done in python. The data used in the model was taken from interviewing a local milk retailer in a small town.

3.1 Data Input and Initialization

The constructed model permits input modelling through two options. Option 1: Historical data is given as an input Excel file and best-fitting distributions are selected by simulation program automatically. Option 2: Distributions (if known) are directly given as input. The input data selected is of option 2 type. The replication length is selected as 120 days as the input distribution were valid for 120 days. After 120 days period, the demand pattern is likely to change due to a change in the season. Information regarding changed distribution is not available. The same stochastic demand values are used to evaluate the policies. This is following the common random numbers variation reduction technique [6]. The following are the simulation parameters validated by retailer:

Table 1. Demand and Shelf Parameters for Each Product

Product	Demand Distribution	Lead Time (L_i)	Shelf Life (S_i)
Milk A	Triangular (140, 144, 147)	1	2
Milk B	Uniform (70, 74)	1	2
Milk C	Uniform (23, 25)	1	2

Table 2. Cost and Pricing Parameters for Each Product

Product	Cost Price (CP_i)	Selling Price (SP_i)
Milk A	28	29
Milk B	24	25
Milk C	11	12

3.2 Discrete Event Simulation

A random number generator is employed to sample values from the specified probability distributions. The simulation is conducted over a replication length of 120 days, with a total of 10 independent replications performed to ensure statistical reliability of the results. The system operates on a daily time step, with each day treated as a discrete event. Key processes—such as inventory updates, order arrivals, demand realization, product expiry, and ordering decisions—are executed sequentially in chronological order within an event-driven framework.

Entities:

- *Product Entity*: Each SKU (e.g., MILK A, MILK B, MILK C).
- *Order Entity*: Represents replenishment batches placed by the retailer.

State Variables:

- $I_i(t)$: On-hand inventory for SKU i at the start of day t .
- $P_i(t)$: Quantity of pipeline (in-transit) orders for SKU i on day t .
- $E_i(t)$: Quantity expired for SKU i at the end of day t .
- $LS_i(t)$: Lost sales (unmet demand) for SKU i on day t .
- $DS_i(t)$: Demand served for SKU i on day t .
- $Q_i(t)$: Quantity ordered for SKU i on day t .

Parameters:

- L_i : Deterministic replenishment lead time for SKU i .
- s_i : Days after which an item expires for SKU i .
- CP_i : Purchase cost of one unit for SKU i .
- SP_i : Selling price of one unit for SKU i .
- S_i : Target base-stock level (optimized using Genetic Algorithm) for SKU i .

Event Types:

- *Order Arrival Event*: Replenishment order arrives, and inventory is updated.

- *Demand Generation Event*: Random or empirical daily demand generated for each SKU.
- *Fulfillment Event*: Demand satisfied from inventory following FIFO logic.
- *Expiry Event*: Items older than shelf life are removed as waste.
- *Ordering Event*: New replenishment placed to restore base-stock level.

Attributes: Each product entity has the following attributes:

- SKU name (e.g., MILK A)
- Arrival day
- Quantity
- Age (days since arrival)

Main Simulation Logic (Pseudocode):

Procedure Main():

Initialize system clock $t \leftarrow 0$

Initialize inventory $I_i(0)$ and pipeline $P_i(0)$ for all SKUs

Initialize event list FEL = {}

Schedule first day event (Day = 1)

for day = 1 to Simulation Horizon (e.g., 120 days) do

Extract next event from FEL

Order Arrival Activity:

Add all arrived quantities to $I_i(t)$ from pipeline orders Demand

Generation Activity:

Generate $D_i(t)$ using empirical or fitted distribution

Fulfillment Activity:

Serve demand using FIFO batches: $DS_i(t) = \min(D_i(t), I_i(t))$

Lost sales: $LS_i(t) = \max(0, D_i(t) - I_i(t))$

Update: $I_i(t) = I_i(t) - DS_i(t)$

Expiry Handling Activity:

Remove expired units older than s_i ;

Compute waste cost: $W_i(t) = CP_i E_i(t)$

Ordering Activity:

$Q_i(t) = \max(0, S_i - [I_i(t) + P_i(t)])$;

Schedule arrival after lead time

Profit Computation Activity:

$Profit_i(t)$

$= (SP_i - CP_i)DS_i(t)$

$- CP_i E_i(t)$

Record daily metrics (profit, loss, expiry, order, inventory)

end for

End Procedure

Warm-Up Consideration: For the distribution-based simulation (option = 2), one warm-up day is simulated to stabilize initial inventory levels before computing statistics. The first day's data is excluded from the final analysis to avoid initialization bias.

Table 3. Python Functions and Variables Used in Simulation Framework.

Function / Variable	Description
run()	Executes the simulation across the demand horizon by iteratively calling step day().
method_run()	Governs the overall simulation flow, including warm-up handling and summary generation.
step_day()	Simulates one operational day involving demand realization, order arrivals, and expiry handling.
warmup_days	Specifies the number of initial days excluded from analysis to remove initialization bias.
optimize_multipliers()	Uses Genetic Algorithm (GA) to optimize base-stock multipliers (S_i) by evaluating multiple candidate solutions.
Sim_run()	Executes a single simulation instance for a GA candidate under identical demand conditions
df["total_profit"].sum()	Computes cumulative profit across the simulation horizon and serves as the GA fitness metric
auto_place_orders	Boolean flag enabling automatic replenishment based on on-hand and pipeline inventory

The Table 3 lists the key functions and variables used from various Python libraries, along with their respective descriptions and roles in the simulation framework.

Simulation Execution (run()): The method run() executes the simulation for the entire demand horizon by following iteration and calling step day(). Parameters like warmup days allow for discarding initial records which does not take part in simulation. To ensure statistical robustness and account for stochastic demand variability, the entire simulation experiment is executed ten times independently within a controlled loop, maintaining identical demand sequences for both policies in each run. The results across these ten replications are subsequently averaged for evaluation and hypothesis testing.

3.3 Policy Implementation

Baseline Fixed-Order

Policy:

In the fixed-order policy, the retailer places a constant replenishment quantity each day, equal to the expected demand for product i . Mathematically, this can be expressed as:

$$Q_i(t) = E[D_i(t)]$$

where $E[D_i(t)]$ represents the expected daily demand irrespective of inventory position. This order quantity remains constant over the simulation horizon and is delivered after a deterministic lead time L_i .

Base-Stock Policy:

In the base-stock policy, the target stock level S_i for each product i is determined as:

$$S_i = E[D_i(t)]k_i$$

where k_i is the optimized base-stock multiplier obtained through the GA-based optimization process. The daily replenishment decision is dynamically computed as:

$$Q_i(t) = S_i - I_i(t)$$

The `auto_place_orders` flag automates the daily replenishment process by updating $Q_i(t)$ in real time based on the observed inventory and pipeline levels. The parameters and functions involved in this policy are implemented as part of the simulation framework and are defined in Table 2.

Simulation-based optimization using GA

This method is generally used to fine-tune the parameters for inventory policy and get the optimal solution [7]. This method has its inspiration from process of natural selection and evolution. Here, multiple possible solution is formed and their performance and iterated it further next generation to get overall best results. There can be multiple optimal results, and for each run answer can be slightly different as depend on demand each days. Here it is done through the function `optimize_multipliers()`.

The Genetic Algorithm (GA) is used to determine the optimal base-stock multipliers k_i within a predefined range of $[0.5, 2.0]$. An initial population of random candidate solutions is created and evaluated by executing the full simulation using `sim.run()` under identical demand scenarios. The fitness of each solution is measured as the total profit, calculated through `df["total profit"].sum()`. A tournament selection strategy is applied to retain the best-performing candidates, followed by uniform crossover to exchange solution traits and Gaussian mutation to introduce diversity and prevent premature convergence. These evolutionary steps repeat across generations until the population converges. Finally, the most profitable set of multipliers k_i is selected, representing the optimized base-stock parameters that achieve the best trade-off between service level and wastage cost.

4 Results

The Genetic Algorithm (GA) efficiently identified the optimal base-stock multipliers for each SKU by averaging the results across 10 independent runs, ensuring robustness against random variations in demand. These optimized multipliers determine the target inventory levels as multiples of the respective SKU's average daily demand, thereby aligning replenishment with realistic consumption patterns.

The optimized values obtained are summarized in Table 4.

Table 4. Optimized Multipliers for SKUs

SKU	Optimized Multiplier (k_i)
Milk A	1.6434
Milk B	1.5555
Milk C	1.4832

Both policies are been compared here through their averages. The GA-optimized policy consistently outperformed the fixed-order approach, as summarized in Table 5.

Table 5. Average Profit Comparison of 10 Runs (in Rs)

SKU	New Policy	Old Policy	% Change
Milk A	143.66	142.99	+0.46%
Milk B	72.03	71.95	+0.11%
Milk C	23.98	23.95	+0.12%
Total	239.67	238.90	+0.33%

The following tests are for the results from 10 runs. The Jarque-Bera test indicates that the profit differences show some normal behavior ($p > 0.05$), while the Breusch–Pagan test confirms homoskedasticity ($p > 0.05$). Consequently, the non-parametric paired t-test was applied. Although the obtained p -value suggests a statistically significant difference between the old and new policies, the average profit increase of only 0.33% indicates that the improvement is economically insignificant. Hence, the GA-optimized policy does not yield a meaningful advantage over the existing policy.

For each SKU and in daily profit terms, the new policy performs better than the existing policy. There is no expiry either in the old policy or in new. Under GA optimized policy, the lost sales have come down to zero, as compared to initially, which gave up to 3. There is a difference in profit in a few days by 1 to 3 Rs with new policy under GA being more profitable, but it is not present on all days. There is a difference in profit difference at some days. The following results for the graph are of a single simulation run.

The actual daily profit of different SKU's was 143, 72 and 24, respectively, and from the model output it's 172.99, 71.95 and. 23.95, respectively which are quite close and thus the model output may be deemed a valid replica of the inventory system.

Table 6. Statistical Test Results for Policy Comparison

Test	Test Statistic p-value	
Normality (Jarque–Bera)	—	0.8139
Homoskedasticity (Breusch–Pagan)	—	0.1470
Selected Test (Paired t-test)	19.9823	<0.001
Average New Profit	239.67	
Average Old Profit	238.90	
Percentage Gain	0.33%	

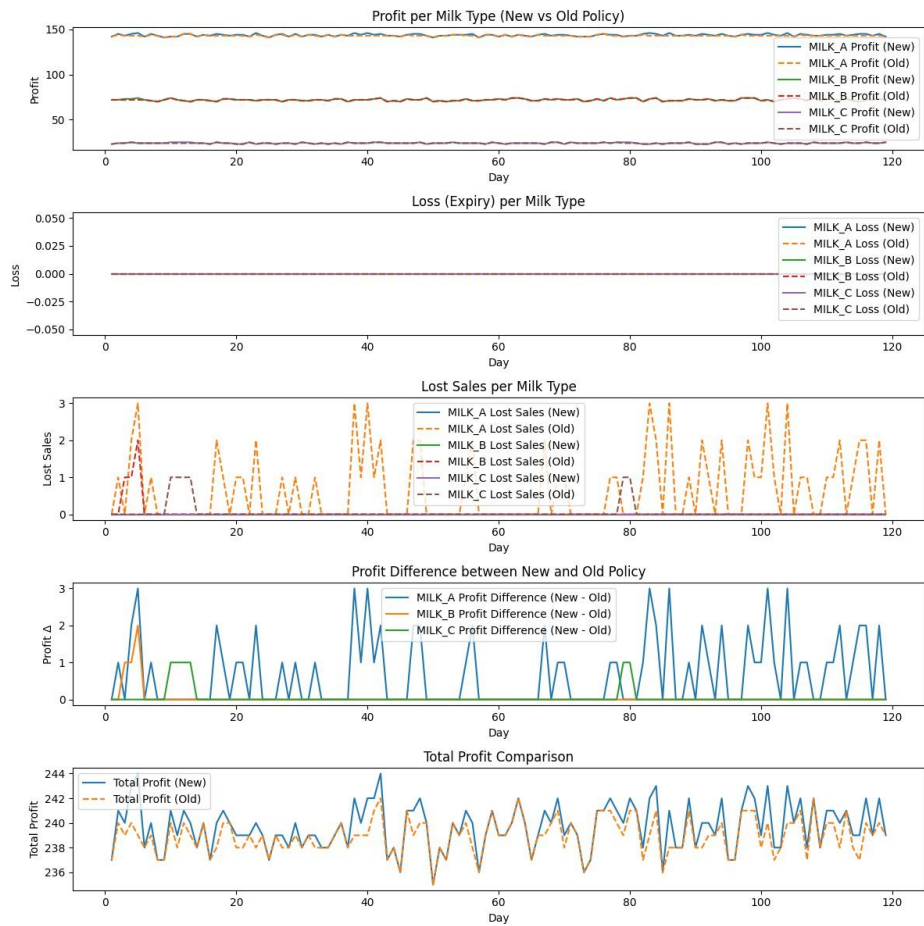


Fig. 1. Comparison of SKU new vs. old policy.

5 Sensitivity Analysis

In this analysis, we attempt to study how the market expansion of the retailer can impact our results. To simulate this we assumed that the milk C's demand followed $U(a,2b)$ with maximum value being doubled. This describes a simulation where milk C, which is a local brand has a higher demand but with increased variability too.

The Genetic Algorithm (GA) efficiently identified the optimal base-stock multipliers for each SKU, which don't show many changes. This simulation is also run 10 times as it was done previously.

Table 7. Optimized Multipliers for SKUs

SKU	Optimized Multiplier (k_i)
Milk A	1.5280
Milk B	1.5059
Milk C	1.6296

The average profit in Milk A and Milk B remains almost the same. However, the average profit has increased to more than 5% using the new policy. Overall, the total profit is increased by more than 1%

Table 8. Average Profit Comparison in Rs.

SKU	New Policy	Old Policy	% Change	% Change
-	Previously	MILK A	143.69	
		142.99	+0.49%	0.46%
MILK. B	72.02	71.94	+0.11%	0.11%
MILK. C	36.74	34.68	+5.94%	0.12%
Total	252.45	249.62	+1.30%	0.33%

The Jarque–Bera test indicates that the profit differences show some normal behavior ($p > 0.05$), while the Breusch–Pagan test confirms homoskedasticity ($p < 0.05$). Consequently, the non-parametric Wilcoxon Signed-Rank was applied. Although the obtained p -value suggests a statistically significant difference between the old and new policies, the average profit increase of only 1.15% indicates that the improvement is economically insignificant. Hence, the GA-optimized policy does not yield a meaningful advantage over the existing policy.

Our results for Figure 2 indicate that there is a high impact on daily profit and lost sales of milk C and optimized policy shows promise. However, the overall

Table 9. Statistical Test Results for Policy Comparison

Test	Test Statistic p-value	
Normality (Jarque–Bera)	—	0.1200
Homoskedasticity (Breusch–Pagan)	—	0.0141
Selected Test (Wilcoxon Signed-Rank)	<0.001	0.001953
Average New Profit in Rs	252.45	
Average Old Profit in Rs	249.62	
Percentage Gain	1.14%	

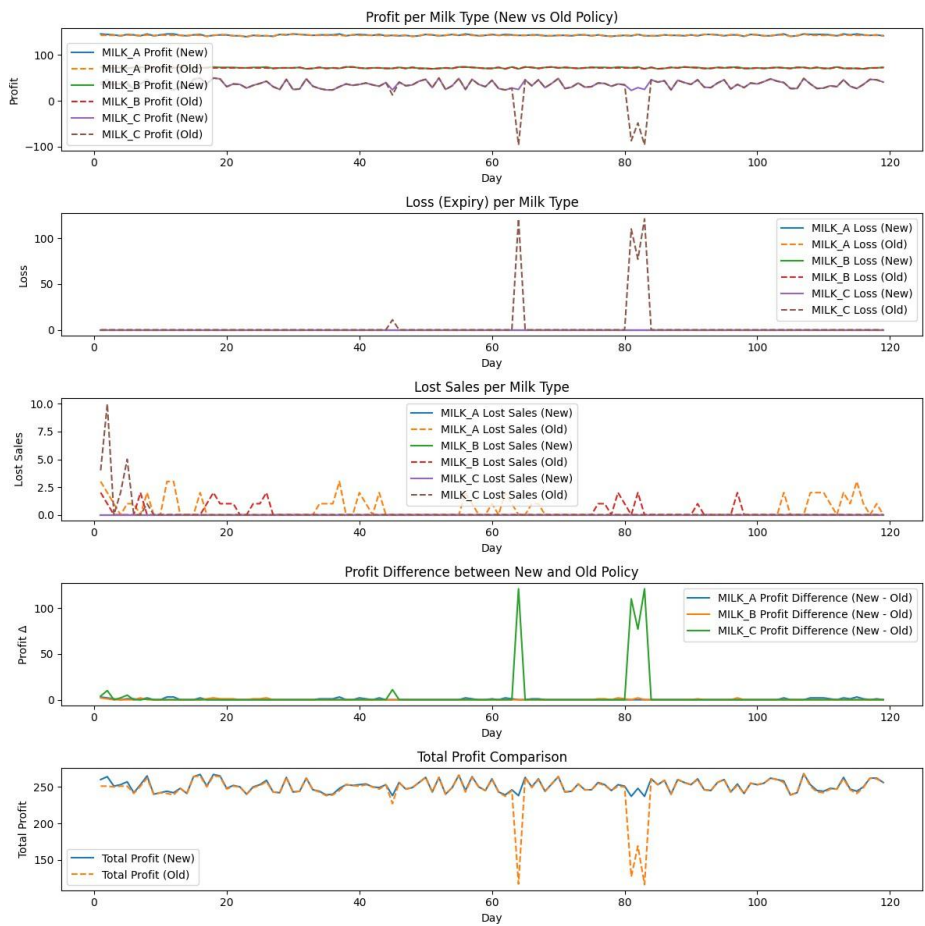


Fig. 2. Comparison of SKU of new vs old policy under changed conditions of Milk C

impact is not high ($< 2\%$). Since contribution of milk C to total profit is not significant. Figure 2 is a representation of a single run only.

However, this sensitivity analysis illustrates that if the retailer were to expand his business and demand for all products were to show greater variability, then the GA optimized base stock policy would meaningfully enhance profitability as compared to the fixed order policy.

6 Conclusion

Existing research suggests that including modern techniques like GA to improve inventory systems can lead to substantial performance enhancement. Hence, in this study, we compared the fixed-order policy used by retailers for perishable goods with a GA-enhanced base-stock policy.

The computational results show very little profit growth (less than 1%), which suggests that such sophisticated methods are not advisable in an unorganized perishable retail sector.

However, as seen from the sensitivity analysis, using such methods in perishable goods, which have much variability, such as fruits and vegetables, can yield significant results for inventory optimization. This can be taken for future work.

Bibliography

- [1] Altiok, T., Melamed, B.: Simulation modeling and analysis with Arena. Elsevier (2010)
- [2] Banks, J.: Discrete event system simulation. Pearson Education India (2005)
- [3] Duong, L.N., Wood, L.C., Wang, J.X., Wang, W.Y.: Transport on-demand in a service supply chain experiencing seasonal demand: Managing persistent backlogs. *International Journal of Operations Research* 14(3), 121–138 (2017)
- [4] Erlenkotter, D.: Ford whitman harris and the economic order quantity model. *Operations Research* 38(6), 937–946 (1990)
- [5] Hiassat, A., Diabat, A., Rahwan, I.: A genetic algorithm approach for location-inventory-routing problem with perishable products. *Journal of manufacturing systems* 42, 93–103 (2017)
- [6] Law, A.M., Kelton, W.D., Kelton, W.D.: Simulation modeling and analysis, vol. 3. McGraw-hill New York (2007)
- [7] Mitchell, M.: An introduction to genetic algorithms. MIT press (1998)
- [8] Nahmias, S.: Myopic approximations for the perishable inventory problem. *Management Science* 22(9), 1002–1008 (1976)
- [9] Silver, E.A., Pyke, D.F., Peterson, R., et al.: Inventory management and production planning and scheduling, vol. 3. Wiley New York (1998)
- [10] Xue, N., Landa-Silva, D., Figueredo, G.P., Triguero, I.: A simulation-based optimisation approach for inventory management of highly perishable food. In: ICORES. pp. 406–413 (2019)

Open Access This chapter is licensed under the terms of the Creative Commons Attribution-NonCommercial 4.0 International License (<http://creativecommons.org/licenses/by-nc/4.0/>), which permits any noncommercial use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons license and indicate if changes were made.

The images or other third party material in this chapter are included in the chapter's Creative Commons license, unless indicated otherwise in a credit line to the material. If material is not included in the chapter's Creative Commons license and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder.

